№3_Архитектура_вычислительных_систем

План

- Виды адресации
  - Команды

Виды адресации

Адресация - это *способ обращения к ячейкам памяти* или регистрам. Различают следующие виды адресации:

1. *Непосредственная* `mov ebx, 5` - `5` имеет непосредственную адресацию. В таком виде адресации *значение операнда кодируется непосредственно в коде* команды.
2. *Регистровая* `mov al, ah` - работаем с регистрами. *Значение операнда содержится в регистре*. Каждый регистр кодируется в коде команды тремя битами.
3. *Косвенная* - *адрес операнда хранится в регистре*.
    1. *Базовая* `mov eax, [ebx]` - команда копирует четыре байта из памяти по адресу записанному в `ebx`. В терминах языка `C`, `ebx` - это указатель на число типа `int`.
    2. *Базовая со смещением* `mov eax, [ecx + 10]` - смещение всегда записывается в байтах.
    3. *Базово-индексная* `mov eax, [edx + esi]` - в базово-индексной адресации эффективный адрес вычисляется сложением двух регистров (в первом регистре адрес начала массива, а во втором смещение относительно начала массива). Значение в квадратных скобках иногда называют эффективным адресом.
    4. *Базово-индексная с масштабированием* `mov eax, [edx, esi * 4]` - существует, чтобы увеличить шаг по

размерности кластера. Масштабный множитель может принимать значения 1, 2, 4, 8.

4. *Прямая* `mov eax, a` - обычно используется для обращения к переменным в сегменте данных, то есть к глобальным переменным.
5. *Неявная* `imul esi` - при неявной адресации команда оперирует с каким-либо операндом, который не фигурирует в самой команде.
6. *Относительная* `lea eax, [eip]` - при относительной адресации адрес вычисляется относительно регистра `eip/rip`.

Для некоторых команд нужно явно указывать размер операнда. Для переопределения размера операнда используют директивы:

- `byte ptr` - 1.
- `word ptr` - 2.
- `dword ptr` - 4.
- `qword ptr` - 8.
- `tbyte ptr` - 10.
- `real4` - 4.
- `real8` - 8.
- `real10` - 10.

Пример:

```
mov [esi], dword ptr 7
```

Напишем программу, которая заполняет массив числами от `0` до `n`:

```
.data
a dd 1024 dup(?)

.code
```

```
mov ebx, offset a
mov esi, 0
mov ecx, 1024

cycle:
     mov [ebx + esi * 4], esi
     add esi, 1
     loop cycle        ; декремент ecx и прыгает на метку
```

Команды

Машинная команда - это *последовательность байтов* в которых хранится информация *о том какую операцию необходимо выполнить процессору, над какими объектами* выполнить операции, *куда сохранить результат*.

В *коде операции кодируется тип команды* (сложение, умножение, деление и тд), сразу после кода операции кодируются *номера регистров, ячейки оперативной памяти, непосредственные числа*.

Операнд - это *объект над которым процессор выполняет* какие-либо *действия*. Чаще всего это регистры процессора, ячейки памяти ил числа.

Если открыть любой 64-битный `.exe` файл в отладчике `x64dbg` и два раза нажать мышкой на команду, то можно её отредактировать.